

Mocha for Unit and Integration NodeJS Testing

- Mocha
- Unit NodeJS Testing with Mocha
- Integration NodeJS Testing with Mocha

Installing Mocha

Mocha is used for testing our **server side** node applications.

Before we get to installing Mocha it's important to discuss why it shouldn't be installed as a global dependency.

Testing frameworks change over time and as such should be added to projects as a developer dependency. Its better to install testing frameworks this way so that changes or revisions to the framework don't break the test code.

Mocha can be installed as a dev dependency using the following command:

```
npm install --save-dev mocha
```

Once Mocha has been installed, we need to set up two test folder locations

YouProjectFolder/unitTest

YouProjectFolder/integrationTest

Installing Mocha

Inside each test folder we also need to create a test file called test.js

yourProjectFolder/unitTest/test.js

yourProjectFolder/integrationTest/test.js

For now add the following base code to each test file

```
var assert = require('assert'); //link in assertion library
describe('Tests for function one', () => {
  describe('Test Case 1 #fnOne()', () => {
    it('should return -1 when the value is not present', () => {
      assert.equal([1,2,3].indexOf(4), -1);
    });
  });
  describe('Test Case #fnOne()', () => {
    it('should return 3 as the value is present', () => {
      assert.equal([1,2,3,4,5].indexOf(4), 3);
    });
  });
});
```

Running Mocha Test

With npm we can run test from the CLI. To do this we need to add the tests to the scripts section of the "package.json" file

```
7     "license": "ISC",
8     "devDependencies": {
9       "mocha": "^6.2.0"
10    },
11    "scripts": {
12      "unitTest": "mocha ./unitTest/test.js",
13      "test": "mocha ./integrationTest/test.js"
14    }
15  }
```

The unit and integration test can now be run using the following commands

npm run-script unitTest

npm run-script test

Assert Libraries and Types

Asserts are used to check the output of a test. Mocha uses nodejs default assert library by default.

However there are several alternatives:

- Should.js
- Expect.js
- Chai
- Better-assert
- Unexpected

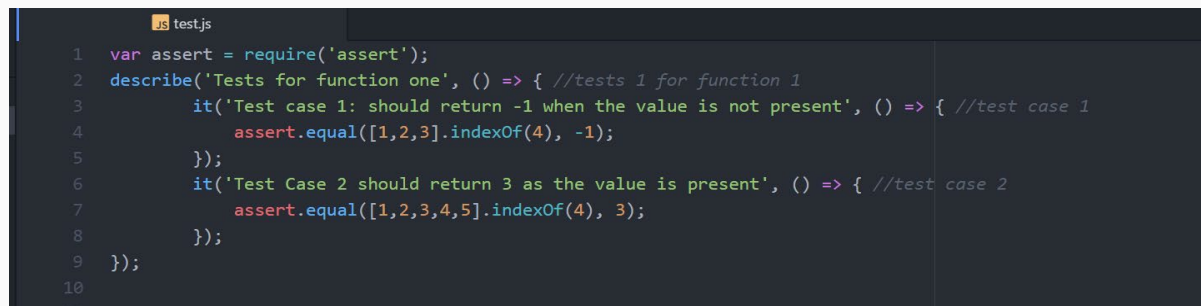
For this course we are going to use node.js default assert library for unit test and Chai for integration tests.

Assert Libraries and Types

Assert Type	Description
<code>assert.ok(value,message)</code>	The most basic type of assert – useful if you need a custom reject or success message. The value is used to pass or fail the test. The message is used to add a description of why the test failed.
<code>assert.equal(value, expected, message)</code>	Is an equivalent to <code>if (value == expected)</code> Cannot be used for to compare objects Can be used for strings, ints, floats, bools The message is used to add a description of why the test failed.
<code>assert.deepEqual(value, expected);</code>	Is used to compare objects and json Checks the value of all keys in an object to see both the value and expected contain the same key/value pares. Does strict comparison and therefore <code>{ a: 1 }</code> and <code>{ a: '1' }</code> are not equal. Value is the generated object Expected is the format and values the value should match
<code>assert.fail(message);</code>	Used to trigger a generic fail and add a custom message. The message is used to add a description of why the test failed.

Formatting Mocha for Unit Test

Mocha test framework is designed to be read similar to human languages.



```
1  var assert = require('assert');
2  describe('Tests for function one', () => { //tests 1 for function 1
3      it('Test case 1: should return -1 when the value is not present', () => { //test case 1
4          assert.equal([1,2,3].indexOf(4), -1);
5      });
6      it('Test Case 2 should return 3 as the value is present', () => { //test case 2
7          assert.equal([1,2,3,4,5].indexOf(4), 3);
8      });
9  });
10
```

The “describe” blocks are used to group tests. The “it” block is used to identify a single test case.

The assert in the middle of each “it” block is used to check the result from the function component or class being tested.

An Toy Example for Unit Testing with Mocha

Suppose we want to test the following function:

```
function(x, y) { return x * y;}
```

First, we need to import the function in the test file.

Rewriting the above function (as below) allows us to easily export the function for testing. To enable your code to be easily tested you may have to refactor it so that it works well with the testing framework.

```
module.exports = function(x, y) {  
  return x * y;  
}
```

An Toy Example for Unit Testing with Mocha

In the test file, we first require module 'assert' and the file '../index' that exports the function being tested. The function is used and the output is tested against an assertion. (Did the correct value get returned or not.) A series of tests can be performed to cater for each usage case of the function. (Correct parameters, Incorrect Parameters, Null cases etc)

```
unitTest > JS test.js > ...
1  var assert = require('assert');
2  var mul = require('../index');
3  describe('Integration Tests for function multiply', () => {
4    describe('Test Case 1 #mul()', () => {
5      it('should return 0 when one of the inputs is 0', () => {
6        assert.equal(mul(2, 0), 0);
7      });
8    });
9    describe('Test Case 2 #mul()', () => {
10     it('should return 4 for 2,2', () => {
11       assert.equal(mul(2, 2), 4);
12     });
13   });
14 });
```

An Toy Example for Unit Testing with Mocha

To run the test, we use **npm run-script unitTest**

```
PS D:\GU3813\w10\ut> npm run-script unitTest

> ut@1.0.0 unitTest D:\GU3813\w10\ut
> mocha ./unitTest/test.js

Integration Tests for function multiply
  Test Case 1 #mul()
    ✓ should return 0 when one of the inputs is 0
  Test Case 2 #mul()
    ✓ should return 4 for 2,2

2 passing (17ms)
```

Formatting Mocha for Integration Test

The first describe block is used to identify a single test. The child describe blocks are used to identify a single test case. Note that each child describe block has a before and after function.

The before function is called before the “it” block is called and should be used to set up any database requirements or other dependencies that the test requires before being run.

The after function is used to clean up any database or dependency changes made before the next test is run. Getting the test back to a base state. We do not one test to pollute another test.

The done call back in the before and after function is used to tell the “it” block that set up or clean up has been finished.

Route Testing with Mocha and Chai

Suppose we have a server file `server.js`, which defines a route `'get/productFind'`. We can test the route by using the `'http'` module.

The major task is usually to check the response body from the server. This example show how to test the status code and how to check the response body.

```
1  var assert = require('assert');
2  var app = require('../server.js');
3  var http = require('http');
4
5  describe('Server test', function() {
6    // The function passed to before() is called before running the test cases.
7    before(function() {console.log("before test");});
8    // The function passed to after() is called after running the test cases.
9    after(function() {console.log("after test");});
10
11    describe('get/productFind', function() {
12      it('should return all products', function() {
13        http.get('http://localhost:3000/productFind', function(response) {
14          // Assert the status code.
15          assert.equal(response.statusCode, 200);
16
17          var body = '';
18          response.on('data', function(d) {body += d;});
19          response.on('end', function() {
20            // Let's wait until we read the response, and then assert the body
21            assert.equal(body, 'Hello Mocha');
22          });
23        });
24      });
25    });
```

Route Testing with Mocha and Chai

It is convenient to use node package 'chai' to write test files for route testing.

Chai is an assertion library.

Chai-http is an addon plugin for the chai assertion library. It allows us to do assertions for common HTTP tasks.

The chunk of code shows us the head part of the test file.

```
let assert = require('assert');
let app = require('../server.js');
let chai = require('chai');
let chaiHttp = require('chai-http');
let should = chai.should();
chai.use(chaiHttp);

describe('Server test', function() {
  // The function passed to before() is called before running the test cases.
  before(function() {
    console.log("before test");
  });

  // The function passed to after() is called after running the test cases.
  after(function() {
    console.log("after test");
  });

  describe('/productFind', () => {
```

Route Testing with Mocha and Chai

We have two tests here. One is for a get method and the other is for a post.

In the second test, the post url is '/productInsert'; and the post body is a form data {'name': 'Kaile', 'id': 3}

As the server will send back the inserted data item, we check if the response body has the properties expected.

```
describe('/productFind', () => {
  it('it should GET all the products', (done) => {
    chai.request(app)
      .get('/productFind')
      .end((err, res) => {
        res.should.have.status(200);
        res.body.should.be.a('array');
        // res.body.length.should.be.eql(2);
        done();
      });
  });
});

describe('/productInsert', () => {
  it('it should insert a doc', (done) => {
    chai.request(app).post('/productInsert').type('form')
      .send({ 'name': 'Kaile', 'id': 3 })
      .end((err, res) => {
        res.should.have.status(200);
        res.body.should.have.property('name');
        res.body.should.have.property('id');
        console.log(res.body);
        done();
      });
  });
});
```

Reading material

1. Mocha Tutorial · GitHub

<https://gist.github.com/soheilhy/867f76f6ea7cab4f8a84>

2. Test a Node RESTful API with Mocha and Chai — Scotch

<https://scotch.io/tutorials/test-a-node-restful-api-with-mocha-and-chai>